Smart Route Finder

Project Title:	Smart route finder
Domain & Technology:	Python / Flask React.js Leaflet SQLite
Team Members & Roll Nos:	1. P.YASHWANTH (Roll No:25MVCSMR0599) 2. SK.RAHMAN (Roll No:25MVCSMR0613) 3. V.RAM CHARAN(Roll No: 25MVCSMR0628) 4.U.PRATHIBHA (Roll No: 25MVCSMR0623)
Department & College:	Department of CSE / AI-ML Malla Reddy Vishwa Vidya Peeth
Project Guide Name:	_____
Academic Year:	2026–2027

2. Abstract & Introduction

Project Overview

Smart Route Finder is a web-based prototype that computes and compares multiple routes between locations, selects the least-cost path, shows viable alternatives, and explains why the chosen route is optimal using three classic algorithms: Dijkstra, Floyd–Warshall, and A*.

Problem Background

Real-world navigation systems must do more than return a single distance: they need to compare paths, reason about travel cost, and make their decisions interpretable to users (e.g., which roads were chosen, why a detour is better, how heuristics guided search).

Proposed Solution

We model the road network as a weighted graph and implement:

- • Dijkstra for single-source shortest paths with non-negative edge costs.
- • Floyd–Warshall for all-pairs shortest paths to support fast queries between many location pairs.
- • A* for goal-directed search using an admissible, consistent heuristic to reduce explored states while guaranteeing optimality.

The UI visualizes the selected path, alternative paths, costs, and algorithm-specific diagnostics (e.g., nodes expanded, heuristic values).

Purpose of the Project

To demonstrate Design and Analysis of Algorithms (DAA) concepts in a usable system that not only computes optimal routes but also makes the algorithmic decisions transparent and comparable.

3. Problem Statement, Objectives & Scope

Problem Statement

Build a route engine that compares multiple paths between locations, calculates the shortest travel cost, displays alternative routes, and explains why the selected route is optimal.

Project Objectives

- • Implement and compare Dijkstra, Floyd–Warshall, and A* on the same graph.

- • Present final results clearly with path details, costs, and explainability artifacts (e.g., expansion order, heuristic estimates).
- • Enable users to see alternative routes and understand trade-offs (distance vs. time vs. tolls, depending on cost mode).

Project Scope

- • Prototype-level system with a small-to-medium synthetic or OpenStreetMap-derived graph.
- • Focus on algorithm correctness, path reconstruction, and explanation—not full-scale production mapping.
- • Web interface for input (source, destination, cost mode), visualization, and result interpretation.

Target Users

- • Students and educators learning shortest-path algorithms.
- • Developers building navigation prototypes who need explainable routing.
- • Analysts comparing algorithm behavior on the same network.

4. Features & Functional Requirements

Main Features

- • Multi-algorithm routing: Run Dijkstra, Floyd–Warshall, and A* on the same query.
- • Alternative routes: Show top-k distinct paths (e.g., via penalty or k-shortest paths approximation).
- • Explainability panel: Display total cost, path sequence, nodes expanded, and (for A*) heuristic values.
- • Map visualization: Render graph and routes using Leaflet.

User Functionalities

- • Select source and destination nodes.
- • Choose cost metric (distance, time, custom weight).
- • Run algorithms individually or in comparison mode.
- • Inspect alternative routes and switch highlights on the map.

CRUD Operations (for demo data)

- • Create: Add/edit nodes (locations) and edges (roads) with weights.
- • Read: List all locations/edges; view graph statistics.
- • Update: Modify edge weights to simulate traffic/tolls.
- • Delete: Remove nodes/edges (with validation to avoid disconnecting the graph in demo mode).

Authentication & Validation

Simple role-based access:

- • Admin: Manage graph data (CRUD).
- • User: Run queries and view results.

Input validation for coordinates, weights (non-negative for Dijkstra/A*), and node existence.

System Requirements

- • Client: Modern browser, internet access for map tiles.
- • Server: Python 3.10+, Flask, SQLite.
- • Memory/CPU: Sufficient for $O(V^3)$ Floyd–Warshall on demo-sized graphs (e.g., $V \leq 200$).

5. Algorithms & Methodology

System Workflow

- User selects source, destination, and cost mode.
- Backend loads graph from SQLite.
- Selected algorithm(s) run:
- - Dijkstra: single-source, non-negative weights.
- - Floyd–Warshall: precomputes all-pairs distances for fast queries.
- - A*: goal-directed search with heuristic h(n).
- Paths are reconstructed and alternatives generated.
- Results (cost, path, diagnostics) returned to frontend for visualization.

Algorithms Used

Dijkstra's Algorithm

- • Maintains tentative costs from source; repeatedly extracts the cheapest unexplored node.
- • Guarantees shortest paths for non-negative edge weights.
- • Path reconstruction via predecessor pointers.

Floyd–Warshall Algorithm

- • Dynamic programming over intermediate vertices to improve all-pairs distance estimates.
- • Time complexity $\Theta(V^3)$; suitable for precomputation on moderate graphs.
- • Can be extended to store "next" or "predecessor" matrices for path reconstruction.

A* Search

- • Evaluation function: f(n) = g(n) + h(n), where g(n) is cost so far and h(n) estimates remaining cost.
- • Admissible heuristic: never overestimates true cost to goal.
- • Consistent heuristic: satisfies triangle-like condition; ensures optimality and efficient expansion.
- • For geographic graphs, straight-line (Euclidean) distance is a common admissible heuristic when edge weights represent distances.

Development Methodology

Iterative prototyping:

- Graph data model & CRUD API.
- Dijkstra + path reconstruction.
- Floyd–Warshall precomputation & queries.
- A* with heuristic and diagnostics.
- Alternative route generation.
- UI polish and explainability views.

Testing after each milestone with synthetic graphs and small real-world extracts.

Data Flow

- Frontend → Backend: Query (source, destination, algorithm, cost mode).
- Backend → DB: Load graph (nodes, edges, weights).
- Backend (Algo Engine): Compute paths, alternatives, diagnostics.
- Backend → Frontend: JSON with path coordinates, costs, and metadata.
- Frontend (Leaflet): Render graph and highlight routes.

6. Technologies & Implementation

Frontend Technologies

- • React for component-based UI.
- • Leaflet for interactive maps and route overlays.
- • State management for algorithm results and map layers.

Backend Technologies

- • Python 3.10+ with Flask REST API.
- • Algorithm modules: dijkstra.py, floyd_warshall.py, astar.py.
- • Explanation engine: formats diagnostics (nodes expanded, heuristic table for A*, intermediate improvements for FW).

Database

SQLite for simplicity and portability.

Tables:

- • nodes(id, name, lat, lon)
- • edges(id, from_node, to_node, weight, cost_type)
- • users(id, username, role, password_hash)

Tools & APIs

- • Postman for API testing.
- • pytest for unit tests on algorithms.
- • Optional: OSMnx or custom scripts to import small OpenStreetMap subgraphs.

Implementation Details

Graph Representation

- • Adjacency list for Dijkstra/A* (efficient neighbor access).
- • Adjacency matrix (or list + DP table) for Floyd–Warshall precomputation.

Path Reconstruction

- • Dijkstra/A*: store parent[node] during relaxation; backtrack from goal.
- • Floyd–Warshall: maintain next[i][j] or pred[i][j] to rebuild paths after DP completes.

Alternative Routes

- • Simple strategy: after finding the shortest path, increase weights on its edges (penalty) and re-run the algorithm to get a different path; repeat for top-k.
- • Store each alternative with its cost and difference from the optimal.

Explainability Artifacts

- • Dijkstra: list of nodes in expansion order, final distances, predecessor chain.
- • Floyd–Warshall: show how distance between a chosen pair improved as intermediate vertices were added (for a demo subset).
- • A*: table of expanded nodes with g(n), h(n), f(n); highlight how heuristic guided search.

7. Results, Testing & Performance

Project Results

- • Correct shortest paths for all three algorithms on test graphs.

- • Alternative routes visibly different on the map with cost deltas.
- • Explainability panels that help users understand why a route was selected.

Test Cases

Functional Tests

- • Single-source to all nodes (Dijkstra) matches known distances on small graphs.
- • All-pairs distances (Floyd–Warshall) match repeated Dijkstra runs for verification.
- • A* returns same optimal cost as Dijkstra when heuristic is admissible and consistent.

Edge Cases

- • Disconnected components: return "no path" with clear message.
- • Multiple equal-cost paths: show at least two alternatives.
- • Zero-weight edges: allowed if non-negative; negative weights disabled for Dijkstra/A*.

Validation Tests

- • Heuristic admissibility check on sample graphs (straight-line ≤ actual shortest path).
- • Consistency check for heuristic (triangle condition) on representative pairs.

Testing Methods

- • Unit tests: algorithm core functions with asserts on distances and paths.
- • Integration tests: full query pipeline (API → DB → algorithm → response).
- • Manual UI tests: verify map rendering, route toggling, and explanation panels.

Performance Analysis

- • Dijkstra: $O((V + E) \log V)$ with priority queue; efficient for single-source queries.
- • Floyd–Warshall: $\Theta(V^3)$; used as a precomputation step for small/medium graphs, enabling $O(1)$ pair queries afterward.
- • A*: Often expands fewer nodes than Dijkstra when heuristic is informative; worst-case similar to Dijkstra.

Measured on a demo graph (e.g., $V \approx 100–200$):

- • FW precomputation completes within seconds.
- • Dijkstra/A* per-query response under a few hundred milliseconds.

Screenshots/Output

(Include in your submitted document:)

- • Map view with optimal route highlighted and alternatives in different colors.

- • Explainability panel showing A* expansion table (g, h, f).
- • FW "progress" view for a small graph (distance matrix snapshots as k increases).

8. Conclusion, Future Scope & References

Project Conclusion

Smart Route Finder successfully demonstrates Dijkstra, Floyd–Warshall, and A* in a single, user-friendly system that not only computes optimal routes but also explains the decisions behind them. By exposing algorithmic state (expansions, heuristics, DP improvements), the prototype turns abstract DAA concepts into observable behavior.

Future Enhancements

- • Support time-dependent edge weights (traffic patterns) and multi-criteria routing (time, distance, tolls).
- • Integrate real map data at larger scale with tiling and caching.
- • Add k-shortest paths algorithms (e.g., Yen's algorithm) for more principled alternative routes.
- • Provide a teaching mode with step-by-step animation of each algorithm.

References / Documentation

• Floyd–Warshall algorithm: all-pairs shortest paths via dynamic programming.

• Dijkstra's algorithm: single-source shortest paths with non-negative weights.

• A* search: informed search with admissible and consistent heuristics.

• Path reconstruction techniques for DP-based APSP.

(Include any textbooks, lecture notes, or library documentation you actually used when preparing your final submission.)